

School Events & Notification Center

Project overview

Build a small project for **school events** (lectures, clubs, workshops, etc.). **Organizers** create and publish events with a **capacity**. **Students** register. If the event is full, the student goes on a **waitlist** (FIFO). When a confirmed student **cancels**, the next waitlisted student can be **promoted** automatically.

When something important happens, the system should **record a “domain event” and enqueue work** so the HTTP request stays fast. A **separate worker process** sends notifications (real email **or** “log only” in development).

User roles

I. Student

- See **published** events and basic details (title, time, place/link, capacity / “full”).
- **Register** (for event) → CONFIRMED if there is capacity, else WAITLISTED with a position (e.g. 1 = next in line).
- **Cancel** own registration if you allow it (simple rule: e.g. always allowed, or until event start).
- See **only own** registrations (never other students’).

II. Organizer

- **Create** and **edit** events in DRAFT, then **publish** (students only see published events).
- Set **capacity** (integer ≥ 1), title, description, start/end (or single datetime), optional location/URL text (plain string - no file upload required).
- **Cancel** an event (e.g. set status so no new registrations; existing registrations handled by a simple rule you document).
- List **registrations + waitlist** for **their** events only (see “ownership” below).

Features

Mandatory features

1. Authentication & roles

- User **registration** and **login**.
- **Role-based access control**: student vs organizer (e.g. **403** when a student tries to create events or when an organizer accesses another organizer’s event).
- Students may see **only their own** registrations.

2. Event catalog (organizer + student views)

- Organizers: create and edit events in **DRAFT** with at least: **title**, **description**, **starts_at** / **ends_at** (or single start time), **capacity** (integer ≥ 1), optional **location or URL** as plain text (no file upload required).
- **Publish** flow: DRAFT → **PUBLISHED** so events become visible to students.

- Students: **list and read** only **PUBLISHED** events; response should allow showing capacity / “full” / counts as you design it.
- **Cancel / close** event (organizer).

3. Registration & waitlist

- Student **registers** for a published event → **CONFIRMED** if capacity available, else **WAITLISTED** with FIFO ordering (e.g. by registered_at).
- **No overbooking** of confirmed seats (e.g. an event with capacity **3** must never have **4** **CONFIRMED** rows, **including** under **concurrent** POST registration requests).
- **At most one** active registration per event.
- Student **cancels** own registration per a **simple rule** (e.g. always allowed, or until event start).
- When a **confirmed** seat is freed: **automatic promotion** of the next student on the waitlist.

4. Organizer operations on registrations

- List **confirmed** registrations for **own** events.
- List **waitlist** in **deterministic order** (FIFO) for **own** events.

5. Async notifications (domain events + queue + worker)

- After the API **persists** registration-related changes **successfully** (e.g. commits the DB transaction or equivalent), it **enqueues jobs** (DB table, Redis, RabbitMQ, etc.); a **separate worker process** processes them.
- At least **three distinct** job/event **types** (e.g. RegistrationConfirmed, RegistrationWaitlisted, WaitlistPromoted)
end-to-end: **API** → **queue** → **worker** → email.

6. REST API coverage

- Implement the **core** REST surface in the **REST endpoints** section of this document (authentication, events including publish/cancel, registrations, registrations/me, organizer lists).
- Proper **HTTP methods and status codes**, **JSON validation errors**, and **authentication** on protected routes.

Optional features

- **GET / PUT /users/me** for profile display and self-service updates.
- **Retries** and **idempotency** for notification jobs (strongly encouraged).
- **Health / readiness** endpoints (/health, /ready) for API and optionally worker.
- **Event cancel**: bulk notification job (EventCancelled) notifying affected users via worker.
- **Simple in-app** notification list (no email) as a second channel.
- **Search/filter** on published events (e.g. by date or text in title—keep minimal).

Architecture

I. Backend (mandatory)

1. Database integration

- Use a relational database (e.g. PostgreSQL, MySQL) for users, events, registrations, etc.

2. RESTful API development

- Endpoints for authentication, events (CRUD + publish + cancel), registrations (create, cancel, list for organizer, me for student), aligned with this document.
- Consistent error responses; authorization checks on every sensitive operation.

3. Event-driven processing

- HTTP API and worker are separate runnable processes (monolithic codebase is fine).
- Producer - the component that **enqueues** jobs when something should happen asynchronously (here: usually the **API** after a successful write)
- Consumer - the component that **pulls** jobs and **does the work** (here: the **worker**). Optional self-study keywords: *producer-consumer pattern, message queue, background job / worker*

Not required: multiple microservices; a single deployable API meets the requirements.

II. Frontend (optional)

1. User interface design

- Responsive, usable UI for browsing published events, register / cancel, and viewing own registrations (student).
- Organizer UI for creating/editing drafts, publish, and viewing registrations + waitlist.

2. Role-based views

- Distinct navigation or screens for student vs organizer (no access to other organizers' events).

3. Preview mode

- Organizer can preview how a published event will appear to students before publishing (or equivalent read-only "student view" of draft).

REST endpoints (guidance)

Paths are examples

Authentication

Method	Path	Description
POST	/register	Create user (role must be restricted: e.g. only student from API, organizers created by seed-your choice, but document it).
POST	/login	Returns token/session.
POST	/logout	If you use sessions.

Events

Method	Path	Description
POST	/events	Organizer: create DRAFT.
GET	/events	Students: published only. Organizer: own drafts + published.
GET	/events/{id}	Details + counts (confirmed / capacity / waitlist size) as appropriate for role.
PUT	/events/{id}	Organizer owns event; only if still editable (define simple rule).
POST	/events/{id}/publish	Draft → published; enqueue EventPublished if you want.
POST	/events/{id}/cancel	Cancel event; enqueue cancellation handling.

Registrations

Method	Path	Description
POST	/events/{id}/registrations	Student registers → confirmed or waitlisted; enqueue domain events.
DELETE	/registrations/{id}	Student cancels own registration; may promote next; enqueue events.
GET	/registrations/me	Student: own list + status + waitlist position.
GET	/events/{id}/registrations	Organizer: confirmed list (ids + names/emails as needed).
GET	/events/{id}/waitlist	Organizer: ordered waitlist.

Profile (optional but useful)

Method	Path	Description
GET	/users/me	Current user profile.
PUT	/users/me	Update own display name / email if allowed.

Notification jobs

Method	Path	Description
GET	/notification-jobs/{id}	Organizer or same user? Simplest: only organizer sees jobs for their event_id; or only internal/debug. Pick one rule and document.
GET	/notification-jobs	Optional list with filters (event_id).

Domain events and message queue - how to use them (guidance)

What is a “domain event” here?

A **domain event** is a **small fact** that already happened in your system, e.g. “registration was confirmed”. It is **not** an HTTP call. You use it to **trigger side effects later**: send email, log, update a read model, etc.

Good habit: events carry **ids and a type**, not full free text from the client.

Example payload (conceptual JSON):

```
{
  "type": "RegistrationConfirmed",
  "event_id": "uuid",
  "user_id": "uuid",
  "registration_id": "uuid",
  "occurred_at": "2026-04-29T12:00:00Z"
}
```

The **worker** loads names/emails from the **database** using those ids. Never trust the client to send “email body” for another user.

What is the “queue”?

Something that stores **jobs** between the API and the worker:

Approach	When to use	Idea
Table as queue (notification_jobs or outbox)	Simplest for beginners	API inserts a row in the same DB transaction as the registration; worker polls WHERE status = 'pending'.
Redis list / Redis Streams	Still small infra	API LPUSH / XADD; worker BRPOP / reads stream.
RabbitMQ	Good “real queue” story	API publishes message; worker consumes.
Kafka	Optional stretch	One topic school-events; producer in API, consumer in worker.

End-to-end flow (recommended pattern)

1. **HTTP handler** validates input and checks auth/role.
2. **Start DB transaction.**
3. **Change business state** (insert/update registration, maybe promote waitlist).
4. **Insert queue row(s)** / publish message with type + ids (same transaction if table queue = **transactional outbox**).

5. **Commit.**
6. Return **200/201** quickly. **Do not** call slow email APIs here.

Which event types are enough for the project?

Teams should implement **at least 3** different type values that result in a queued job, for example:

- RegistrationConfirmed - student got a seat.
- RegistrationWaitlisted - student is on waitlist (optional email; can be same template category).
- RegistrationCancelled - user cancelled (optional: email to user).
- WaitlistPromoted - student moved from waitlist to confirmed (important “happy path”).
- EventPublished - optional notify nobody or log only.
- EventCancelled - optional: enqueue **one job per registered user** in a loop **inside the transaction** (many rows) **or** one job “bulk_cancel” with event_id and let worker query users (simpler: one bulk job).

Idempotency

Workers **retry**; the same logical thing might be processed twice. Safe approaches for beginners:

- Store idempotency_key on the job (e.g. registration_id + ":" + "RegistrationConfirmed" unique), **or**
- Worker checks DB: “if notification already logged for this registration + type, skip”.